

ユニット 7・第10-12週

スレッドと並行性

📅 第10-12週・日程は未定 </> 演習 TD6（解答 + 生成AIチャレンジ） ~ アニメーション 5 本

演習 TD6・解答付き

スレッドと並行性

複数のスレッドが1つのプロセスのメモリを共有する：速くて便利だが、プログラミングで最も厄介なバグの源でもある。この演習ではそれをわざと起こし、ミューテックスとセマフォで直す。



課題：実習レポート

この実習は短い LaTeX レポート（必要なことだけ）で提出します。扱う内容、テンプレート、例、Prism のガイドは「課題」タブにあります。生成 AI は申告すれば利用できます。

演習問題

- | | |
|------------------------|---|
| 1 スレッドを作成して join する | 4 食事する哲学者 |
| 2 チケットの売り手：競合と 2 つの解決策 | 5 探究：スレッドのコスト、x86 での Peterson、futex（新規） |
| 3 生産者と消費者 | |

TD6 ・ 1

● 解答済み・テスト済み

スレッドを作成して join する

課題。 `pthread_create` で 8 つのスレッドを作り、 `start` を実行させて（引数で渡した）プロセスの pid を表示させてください。メインスレッドは `pthread_join` で全員を待ちます。
`start` にグローバル変数とローカル変数を 1 つずつ加え、それを増やして、アドレスと値を表示してください。

解答



C tds/td6/threads.c

```

/*
 * TD6 - exercise 1: create 8 threads, wait for them, and compare a global
 and a local variable.
 * All threads share the global `shared` (same address) but each has its own
 `local` (on its
 * own stack): the addresses printed show it. `shared++` is a race: see
 exercise 2.
 * Build: gcc -pthread (the Makefile already does it).
 */
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define NTHREADS 8

static int shared = 0;

static void *start(void *arg)
{
    pid_t pid = (pid_t)(intptr_t)arg;      /* the pid travels inside the
void * */
    int local = 0;

    shared++;
    local++;
    printf("thread %lu of process %d: shared[%p] = %d, local[%p] = %d\n",
        (unsigned long)pthread_self() % 100000, pid, (void *)&shared,
shared, (void *)&local, local);
    return NULL;                          /* same as pthread_exit(NULL) */
}

int main(void)
{
    pthread_t tids[NTHREADS];
    pid_t pid = getpid();

    for (int i = 0; i < NTHREADS; i++) {
        int rc = pthread_create(&tids[i], NULL, start, (void *)
(intptr_t)pid);

```

```
    exit_if(rc != 0, "pthread_create"); /* pthread functions return the
    error code */
}
for (int i = 0; i < NTHREADS; i++)
    exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");

printf("process %d: all %d threads finished, shared = %d\n", pid,
NTHREADS, shared);
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./threads
thread 57632 of process 357789: shared[0x650ff476602c] = 1,
local[0x7626e31fee74] = 1
thread 79520 of process 357789: shared[0x650ff476602c] = 4,
local[0x7626e19fbe74] = 1
thread 19968 of process 357789: shared[0x650ff476602c] = 5,
local[0x7626dbffee74] = 1
thread 64928 of process 357789: shared[0x650ff476602c] = 2,
local[0x7626e29fde74] = 1
thread 72224 of process 357789: shared[0x650ff476602c] = 3,
local[0x7626e21fce74] = 1
thread 86816 of process 357789: shared[0x650ff476602c] = 6,
local[0x7626e11fae74] = 1
thread 94112 of process 357789: shared[0x650ff476602c] = 7,
local[0x7626e09f9e74] = 1
thread 27264 of process 357789: shared[0x650ff476602c] = 8,
local[0x7626db7fde74] = 1
process 357789: all 8 threads finished, shared = 8
```

すべてのスレッドが同じ **pid** (1つのプロセスだから) と、**shared** の同じアドレスを表示しますが、**local** はそれぞれ別のアドレスにあります：各スレッドが自分のスタックを持つからです。**shared** の値は順不同で出て、重複することさえあります：**shared++** はアトミックではありません。

チケットの売り手：競合と2つの解決策

課題。学部のパティーのために、8人の学生（スレッド）がグローバルなカウンタのチケットを並行して売ります。残っている間は `sched_yield` を呼び、1枚引いて自分の販売数を数え、最後に売った枚数を `pthread_exit` で返し、メインスレッドが合計します。量を増やして試してください。何が起きますか？クリティカルセクションはどこですか？セマフォとミューテックスで保護してください。

解答

C tds/td6/tickets_race.c

```
static volatile int tickets = 1000;    /* volatile only prevents caching in a
register:

                                         it does NOT make the accesses
atomic */

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    while (tickets > 0) {                /* critical section starts here ...
*/
        sold++;
        sched_yield();                  /* do not touch this line
*/
        tickets--;                      /* ... and ends here
*/
    }
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;     /* same as pthread_exit((void *)sold)
*/
}
```

実際の出力

```
$ ./tickets_race 100000 | tail -3
[5] I sold 14026 tickets
[7] I sold 14753 tickets
103195 tickets sold out of 100000 (tickets = -7)
```

実際の枚数より多くのチケットが売れ、カウンタは負になって終わります。

クリティカルセクションは `tickets > 0` から `tickets--` までの全体です：確認と引き算の間に別のスレッドが最後のチケットを売ることがあり、`tickets--` 自体も「読む・引く・書く」の3手順でインターリーブしうるからです。

C tds/td6/tickets_mutex.c

```
static int tickets = 1000;
static pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    for (;;) {
        pthread_mutex_lock(&lock);
        if (tickets ≤ 0) {                               /* checked with the lock held */
            pthread_mutex_unlock(&lock);
            break;
        }
        sold++;
        sched_yield();
        tickets--;
        pthread_mutex_unlock(&lock);
    }
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *) (intptr_t)sold;
}
```

実際の出力

```
$ ./tickets_mutex 100000 | tail -3
[3] I sold 11577 tickets
[2] I sold 11338 tickets
100000 tickets sold out of 100000 (tickets = 0)
```

C tds/td6/tickets_sem.c

```
static sem_t available;

static void *seller(void *arg)
{
    int id = (int)(intptr_t)arg;
    long sold = 0;

    while (sem_trywait(&available) == 0) { /* took one ticket */
        sold++;
        sched_yield();
    }
    exit_if(errno != EAGAIN, "sem_trywait"); /* EAGAIN = no tickets left,
    anything else is a bug */
    printf("[%d] I sold %ld tickets\n", id, sold);
    return (void *)(intptr_t)sold;
}
```

実際の出力

```
$ ./tickets_sem 100000 | tail -3
[6] I sold 14469 tickets
[2] I sold 11537 tickets
100000 tickets sold out of 100000 (tickets = 0)
```

ミューテックスでは、確認と引き算をロックを持ったまま行います（中で確認をやり直します：ロックを取る前に読んだ値は古いかもしれないからです）。セマフォならさらに良い設計になります：セマフォそのものがチケットのカウンタで、`sem_trywait` が1つの分割できない操作で「確かめて取る」のです。

⚠ 修正版

引き継いだミューテックス版はコンパイルできず（`nbtickets_venduss`）、ロックの外でカウンタを読んでいた。

アニメーション A7.1・競合状態 →

生産者と消費者

課題。(a) 生産者は配列 **A** の全セルに1を足し、消費者は全セルを合計します。それぞれ **LOOP** 回です。配列全体に排他的にアクセスしながら**交代**で動く2つのスレッドにしてください。(b) 今度は生産者1つと消費者4つです：生産された値はちょうど1回ずつ消費されなければなりません。どんな新しいクリティカルセクションが現れますか？

解答

C tds/td6/prodcons.c

```
/*
 * TD6 - exercise 3 (a): one producer and one consumer take turns on the
 * WHOLE array.
 * The producer adds 1 to every cell, then the consumer sums every cell; LOOP
 * rounds.
 * Expected sum: SIZE * (1 + 2 + ... + LOOP) = SIZE * LOOP * (LOOP + 1) / 2.
 *
 * Two semaphores implement the turn-taking ("baton passing"):
 * can_produce = 1 at start (the producer goes first), can_consume = 0.
 * The critical section is the traversal of the array: while one thread walks
 * it, the
 * other must not touch it. With the two semaphores, at most one of them is
 * inside.
 */
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define SIZE 10000
#define LOOP 10

static int A[SIZE];
static long sum = 0;
static sem_t can_produce, can_consume;

/* "Opaque" functions of the statement: they must not be modified. */
static int consume(int *value) { usleep(1); return *value; }
```

```
static void produce(int v, int *value) { usleep(1); *value += v; }

static void *producer(void *arg)
{
    (void)arg;
    for (int i = 0; i < LOOP; i++) {
        sem_wait(&can_produce);
        /* my turn */
        for (int j = 0; j < SIZE; j++)
            produce(1, &A[j]);
        printf("producer: round %d written\n", i + 1);
        sem_post(&can_consume);
        /* your turn */
    }
    return NULL;
}

static void *consumer(void *arg)
{
    (void)arg;
    for (int i = 0; i < LOOP; i++) {
        sem_wait(&can_consume);
        for (int j = 0; j < SIZE; j++)
            sum += consume(&A[j]);
        printf("consumer: sum after round %d = %ld\n", i + 1, sum);
        sem_post(&can_produce);
    }
    return NULL;
}

int main(void)
{
    pthread_t p, c;

    exit_if(sem_init(&can_produce, 0, 1) == -1 || sem_init(&can_consume, 0,
0) == -1, "sem_init");
    exit_if(pthread_create(&p, NULL, producer, NULL) != 0, "pthread_create");
    exit_if(pthread_create(&c, NULL, consumer, NULL) != 0, "pthread_create");
    exit_if(pthread_join(p, NULL) != 0 || pthread_join(c, NULL) != 0,
"pthread_join");

    long expected = (long)SIZE * LOOP * (LOOP + 1) / 2;
    printf("sum = %ld, expected %ld: %s\n", sum, expected, sum == expected ?
"OK" : "WRONG");
    sem_destroy(&can_produce);
    sem_destroy(&can_consume);
}
```

```
    return sum == expected ? EXIT_SUCCESS : EXIT_FAILURE;
}
```

実際の出力

```
$ ./prodcons | tail -4
consumer: sum after round 9 = 450000
producer: round 10 written
consumer: sum after round 10 = 550000
sum = 550000, expected 550000: OK
```

(a) 2つのセマフォがリレーのバトンのように順番を渡します: `can_produce` は1、`can_consume` は0から始まります。各スレッドは自分のセマフォを待ち、配列をたどり、相手を有効にします。期待される合計は $\text{SIZE} \times (1 + 2 + \dots + \text{LOOP}) = 10,000 \times 55$ です。

C tds/td6/prodcons_multi.c

```
/*
 * TD6 - exercise 3 (b): one producer, N consumers, and a bounded buffer (the
 * classic of the
 * lecture). The producer puts ITEMS values in a circular buffer of BUF
 * cells; the consumers
 * take them out and add them to a shared sum. Every value must be consumed
 * exactly once.
 *
 * empty = number of free cells (starts at BUF) - the producer waits on
 * it
 * full  = number of used cells (starts at 0) - the consumers wait on
 * it
 * mutex = protects the `out` index shared by the consumers (new critical
 * section!)
 * sum_lock protects the shared sum
 * To stop, the producer sends one "poison pill" (value -1) per consumer.
 * ./prodcons_multi [consumers] [items] (default 4 consumers, 100000
 * items)
 */
#include <pthread.h>
#include <semaphore.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

#define BUF 16
```

```
#define POISON (-1)

static int buffer[BUF];
static int in = 0, out = 0;           /* in: producer only; out: shared
by consumers */
static sem_t empty, full;
static pthread_mutex_t out_lock = PTHREAD_MUTEX_INITIALIZER;
static pthread_mutex_t sum_lock = PTHREAD_MUTEX_INITIALIZER;
static long long sum = 0;
static int nconsumers = 4, items = 100000;

static void *producer(void *arg)
{
    (void)arg;
    for (int v = 1; v ≤ items + nconsumers; v++) {
        int value = v ≤ items ? v : POISON;
        sem_wait(&empty);           /* wait for a free cell */
        buffer[in] = value;         /* only one producer: no lock
needed for `in` */
        in = (in + 1) % BUF;
        sem_post(&full);           /* one more item available */
    }
    return NULL;
}

static void *consumer(void *arg)
{
    long mine = 0, count = 0;
    for (;;) {
        sem_wait(&full);
        pthread_mutex_lock(&out_lock); /* two consumers must not take
the same cell */
        int value = buffer[out];
        out = (out + 1) % BUF;
        pthread_mutex_unlock(&out_lock);
        sem_post(&empty);
        if (value == POISON)
            break;
        mine += value;
        count++;
    }
    pthread_mutex_lock(&sum_lock); /* add the partial sum once, not
per item */
    sum += mine;
    pthread_mutex_unlock(&sum_lock);
}
```

```

    printf("consumer %d: %ld items\n", (int)(intptr_t)arg, count);
    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc > 1) nconsumers = atoi(argv[1]);
    if (argc > 2) items = atoi(argv[2]);
    pthread_t prod, cons[nconsumers];

    exit_if(sem_init(&empty, 0, BUF) == -1 || sem_init(&full, 0, 0) == -1,
"sem_init");
    exit_if(pthread_create(&prod, NULL, producer, NULL) != 0,
"pthread_create");
    for (int i = 0; i < nconsumers; i++)
        exit_if(pthread_create(&cons[i], NULL, consumer, (void *) (intptr_t)i)
!= 0, "pthread_create");
    exit_if(pthread_join(prod, NULL) != 0, "pthread_join");
    for (int i = 0; i < nconsumers; i++)
        exit_if(pthread_join(cons[i], NULL) != 0, "pthread_join");

    long long expected = (long long)items * (items + 1) / 2;
    printf("sum = %lld, expected %lld: %s\n", sum, expected, sum == expected
? "OK" : "WRONG");
    return sum == expected ? EXIT_SUCCESS : EXIT_FAILURE;
}

```

実際の出力

```

$ ./prodcons_multi 4 100000
consumer 2: 23918 items
consumer 3: 26423 items
consumer 1: 23037 items
consumer 0: 26622 items
sum = 5000050000, expected 5000050000: OK

```

(b) 古典的な有限バッファの問題： `empty` は空きセル、 `full` は埋まったセルを数えます。消費者が複数になると新しいクリティカルセクションが現れます： `out` の添字で、ミューテックスで守ります（さもないと2つの消費者が同じセルを取ってしまいます）。各消費者の合計は別のミューテックスのもとで1回だけ足されます。終わらせるため、生産者は消費者ごとに「毒薬」(-1)を1つ送ります。



一貫性を修正

課題文は `producteur / consommateur` と言い、引き継いだ解答は `lecteur / ecrivain` と言っていました。そのうち1つは「配列全体への排他的アクセス」を満たさず、複数消費者の版は `sem_trywait + usleep` でビジーウェイトしていました。今は課題文、コード、名前がそろっています。

TD6 ・ 4

● 解答済み・テスト済み（新規）

食事する哲学者

課題。8 人の哲学者が丸いテーブルに座り、隣どうしの間に箸が1本ずつあります。食べるには右と左の箸が必要です（箸ごとに1つのミューテックス）。全員がまず右、次に左の箸を取り、「考える・取る・食べる・置く」を繰り返します。何が起きますか？ それを避ける変更を提案してください。

解答

C tds/td6/philosophers.c

```
/*
 * TD6 - exercise 4: the dining philosophers. N philosophers, N chopsticks
 (one mutex each).
 * Every philosopher takes the RIGHT chopstick, then the LEFT one.
 * DEMO_DEADLOCK (Makefile target philosophers_deadlock): everybody may
 hold the right
 * chopstick and wait forever for the left one: circular wait = deadlock. A
 pause between
 * the two lock operations makes it happen almost every time.
 * default (solution): break the circular wait by ordering the resources -
 every
 * philosopher takes the chopstick with the SMALLER number first. The last
 philosopher
 * (whose right chopstick is N-1 and left is 0) therefore starts with the
 left one.
 * ./philosophers [meals]    (default 5 meals each)
 */
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
```

```
#include <unistd.h>

#include "utils.h"

#define N 8

static pthread_mutex_t chopstick[N];
static int meals = 5;

static void think(int id)
{
    printf("%2d: thinking\n", id);
    usleep((useconds_t)(random() % 2000));
    printf("%2d: hungry\n", id);
}

static void eat(int id)
{
    printf("%2d: eating\n", id);
    usleep((useconds_t)(random() % 2000));
    printf("%2d: full\n", id);
}

static void *philosopher(void *arg)
{
    int id = (int)(intptr_t)arg;
    int right = id, left = (id + 1) % N;
#ifdef DEMO_DEADLOCK
    int first = right, second = left;
#else
    int first = right < left ? right : left;    /* global order on the
chopsticks */
    int second = right < left ? left : right;
#endif
    for (int m = 0; m < meals; m++) {
        think(id);
        pthread_mutex_lock(&chopstick[first]);
#ifdef DEMO_DEADLOCK
        usleep(20000);    /* everybody grabs the
first one... */
#endif
        pthread_mutex_lock(&chopstick[second]);
        eat(id);
        pthread_mutex_unlock(&chopstick[second]);
        pthread_mutex_unlock(&chopstick[first]);
    }
}
```

```

    }
    printf("%2d: done after %d meals\n", id, meals);
    return NULL;
}

int main(int argc, char *argv[])
{
    pthread_t tids[N];

    if (argc > 1)
        meals = atoi(argv[1]);
    setvbuf(stdout, NULL, _IOLBF, 0);    /* line by line, even into a pipe:
see who is stuck */
    srand(2026);
    for (int i = 0; i < N; i++)
        exit_if(pthread_mutex_init(&chopstick[i], NULL) != 0,
"pthread_mutex_init");
    for (int i = 0; i < N; i++)
        exit_if(pthread_create(&tids[i], NULL, philosopher, (void *)
(intptr_t)i) != 0, "pthread_create");
    for (int i = 0; i < N; i++)
        exit_if(pthread_join(tids[i], NULL) != 0, "pthread_join");
    printf("dinner finished: %d philosophers x %d meals\n", N, meals);
    return EXIT_SUCCESS;
}

```

実際の出力

```

$ timeout 3 ./philosophers_deadlock 3 | tail -8; echo "exit status:
${PIPESTATUS[0]} (124 = still blocked after 3 s)"
7: hungry
2: hungry
3: hungry
1: hungry
0: hungry
6: hungry
4: hungry
5: hungry
exit status: 124 (124 = still blocked after 3 s)

```

実際の出力

```
$ ./philosophers 3 | tail -3
7: full
7: done after 3 meals
dinner finished: 8 philosophers x 3 meals
```

全員がまず右を取る場合、8人が同時にそうするだけで、各自が隣の人の持つ箸を待つこととなります：**デッドロック**です。実際の出力では8人とも「空腹」で、**timeout** が終了させるまでプログラムは止まったままです。Coffman の 4 条件（相互排除、保持と待機、横取り不可、循環待ち）がそろっています。解決策は**循環待ち**を崩すことです：箸に番号を付け、各哲学者は番号の小さいほうを先に取ります。すると最後の哲学者は左を先に取ることで、循環が閉じられません。

[アニメーション A7.3・デッドロック →](#)

TD6 ・ 5

● 新規・解答済み・テスト済み

探究：スレッドのコスト、x86 での Peterson、futex

課題。スライドの 3 つの主張を確かめる（または否定する）3 つの測定：(1) **create_cost** : スレッドの作成はプロセスの作成より安いのか？（スライド 136）。(2) **peterson** : Peterson のアルゴリズムは「正しい」（スライド 152）...実際のプロセッサでも？ **peterson_fence** と比べてください。(3) **strace -c** の下での **lock_cost** : ミューテックスは競合があるときとないときで何回システムコールを行うか？（スライド 160~162）。

解答と実際の出力

C tds/td6/create_cost.c

```
/*
 * TD6 - exploration 1: how much does creating a thread cost compared with a
 * process? (original slide 136)
 * Creates and waits for N threads (pthread_create + pthread_join) and N
 * processes (fork + waitpid, the
 * child exits at once) and prints the average time of each.
 * ./create_cost [N]      default: 2000
 */
#include <pthread.h>
#include <stdio.h>
```

```
#include <stdlib.h>
#include <sys/wait.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static double now(void)
{
    struct timespec ts;
    exit_if(clock_gettime(CLOCK_MONOTONIC, &ts) == -1, "clock_gettime");
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static void *nothing(void *arg)
{
    return arg;
}

int main(int argc, char *argv[])
{
    int n = argc > 1 ? atoi(argv[1]) : 2000;

    double t0 = now();
    for (int i = 0; i < n; i++) {
        pthread_t t;
        int rc = pthread_create(&t, NULL, nothing, NULL);
        exit_if(rc != 0, "pthread_create");
        exit_if(pthread_join(t, NULL) != 0, "pthread_join");
    }
    double thread_us = (now() - t0) * 1e6 / n;

    t0 = now();
    for (int i = 0; i < n; i++) {
        pid_t pid = fork();
        exit_if(pid == -1, "fork");
        if (pid == 0)
            _exit(0);
        exit_if(waitpid(pid, NULL, 0) == -1, "waitpid");
    }
    double fork_us = (now() - t0) * 1e6 / n;

    printf("%d threads:   %6.1f us per pthread_create + pthread_join\n", n,
        thread_us);
    printf("%d processes: %6.1f us per fork + waitpid\n", n, fork_us);
}
```

```
printf("a process costs %.1f times a thread\n", fork_us / thread_us);
return EXIT_SUCCESS;
}
```

実際の出力

```
$ ./create_cost 2000
2000 threads:      41.2 us per pthread_create + pthread_join
2000 processes:    248.8 us per fork + waitpid
a process costs 6.0 times a thread
```

スレッドはページテーブルもディスクリプタもコピーせず、スタックとタスク構造体を確保するだけです。Linux ではどちらも `clone` で作られ、違うのは何を共有するかです。

C tds/td6/peterson.c

```
/*
 * TD6 - exploration 2: Peterson's algorithm on a real multicore x86
 * (original slides 152 and 154).
 * Two threads increment a shared counter N times each, protecting the
 * increment with Peterson's lock.
 * The algorithm is correct on paper, but it "assumes memory accesses are
 * sequentially ordered per thread"
 * (slide 154). Without a fence, the x86 store buffer lets each thread read
 * the other's flag BEFORE its own
 * store is visible: both enter the critical section and increments are lost.
 * With a full fence
 * (mfence, via __atomic_thread_fence) the algorithm works. Compiled twice:
 * peterson (no fence) and
 * peterson_fence (-DFENCE).
 * ./peterson [N]      default: 2000000
 */
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#include "utils.h"

static volatile int flag[2];
static volatile int turn;
static volatile long counter;
static long n;

static void *worker(void *arg)
{
```

```

int me = (int)(long)arg, other = 1 - me;
for (long i = 0; i < n; i++) {
    flag[me] = 1;
    turn = other;
#ifdef FENCE
    __atomic_thread_fence(__ATOMIC_SEQ_CST);    /* the stores must be
visible before the loads */
#endif
    while (flag[other] && turn == other)
        ;                                       /* busy waiting */
    counter = counter + 1;                       /* critical section */
    flag[me] = 0;
}
return NULL;
}

int main(int argc, char *argv[])
{
    n = argc > 1 ? atol(argv[1]) : 2000000;
    pthread_t t[2];
    for (long i = 0; i < 2; i++)
        exit_if(pthread_create(&t[i], NULL, worker, (void *)i) != 0,
"pthread_create");
    for (int i = 0; i < 2; i++)
        exit_if(pthread_join(t[i], NULL) != 0, "pthread_join");
#ifdef FENCE
    const char *variant = "with fence";
#else
    const char *variant = "no fence";
#endif
    printf("Peterson %-10s: counter = %ld, expected %ld, lost %ld\n",
variant, counter, 2 * n, 2 * n - counter);
    return EXIT_SUCCESS;
}

```

実際の出力

```
$ for i in 1 2 3 4 5; do ./peterson; done; for i in 1 2 3 4 5; do
./peterson_fence; done
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson no fence : counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
Peterson with fence: counter = 4000000, expected 4000000, lost 0
```

フェンスなしでは、一部の実行で増分が失われます：2つのスレッドが同時にクリティカルセクションにいたのです。x86は各書き込みを**バッファ**に残し、自分のメモが見える前に、あるスレッドがもう一方のメモを読めてしまいます（TSOモデル）。

`__atomic_thread_fence`（`mfence` 命令）がそれを防ぎます。スライド 154 が警告し、アニメーション A7.5 が状態ごとに検証していることです。

C tds/td6/lock_cost.c

```
static void *worker(void *arg)
{
    for (long i = 0; i < n; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return arg;
}
```

実際の出力

```
$ ./lock_cost 1; ./lock_cost 8
1 thread(s) x 1000000 lock/unlock: counter = 1000000, 28.2 ns per lock/unlock
8 thread(s) x 1000000 lock/unlock: counter = 8000000, 184.2 ns per
lock/unlock
```

実際の出力

```
$ strace -f -c -e trace=futex ./lock_cost 1 100000; strace -f -c -e
trace=futex ./lock_cost 8 100000
1 thread(s) x 100000 lock/unlock: counter = 100000, 48.4 ns per lock/unlock
0.00    0.000000    0    1    futex
8 thread(s) x 100000 lock/unlock: counter = 800000, 85.5 ns per lock/unlock
100.00  0.223730    27    8013    3116 futex
```

競合がなければ、100,000 回の `lock / unlock` でも `futex` の呼び出しは**ゼロ**です（唯一のものは `join` のもの）：空いているミューテックスはユーザ空間のアトミック命令で取れます。スレッドが待つ必要があるときだけカーネルに入ります。スライド 160 の「限定的なビジーウェイト」を、Linux の `futex` が実現したものです。

検証を見る → A7.5 牛乳問題

チェックリスト

生成AIチャレンジ・哲学者：グラフでデッドロックを検出する



目標。 `philosophers.c` に計測を加えて誰が誰を待っているかを記録し、待ちグラフの循環を自動で検出する。2つの解決策（全体順序と「給仕」セマフォ）を比べる。

最初のプロンプト。 好きなツール（Claude、ChatGPT、Gemini、Copilot、Codex...）で、演習の解答付きバージョンをコンテキストとして使ってください。まず計画を求め、各ステップを確認してから受け入れましょう。

Prompt

ミューテックスを使った食事する哲学者のプログラムがあります [philosophers.c を貼り付け]。次を手伝ってください：(1) 100 ms ごとに「哲学者 i は j が持つ箸を待っている」というグラフを作って循環を検出する見張りスレッドを加える。(2) 給仕による解決策（N-1 人だけ座らせるセマフォ）を実装し、全体順序による解決策と 1 秒あたりの食事回数で比べる。(3) 各解決策を、それが崩す Coffman の条件と結び付ける。コードを書く前に、それぞれが 4 条件のどれを崩すかを私に質問してください。

結果の確かめ方

- 見張りスレッドが `philosophers\_deadlock` の循環を検出し、循環に含まれる 8 人の哲学者を表示する。
- どちらの解決策も必ず終わる（20 回繰り返す）。
- 1 秒あたりの食事回数は AI ではなく測定から出したものである。

AI の利用を申告しましょう：使ったツール、主なプロンプト、自分で確認したこと、修正したこと。大切なのは、すべての判断を説明し検証できることです。